

Lab on OpenSSL

1 Introduction

The primary problem of the Internet is security and it is not the main concern to the designers of Internet. However, the tremendous growth of the e-commerce, social networks, mobile wallets, and etc., are carrying lot sensitive information. Insecure network architectures cause too many security and privacy issues. Due to the insecure communication Internet is need to add secure mechanisms to protect user's sensitive information. Now a day, all Internet users rely on the cryptosystems to provide confidentiality, integrity and availability. In this lab, primary focus is on the secure communication protocols like secure socket layer (SSL) which is used in all the modern web browsers and other applications.

2. Objective of the LAB

In this laboratory, three main observations are required to investigate and details are in the following.

Observation 1: The confidentiality and Secure Socket Layer (SSL) : Confidentiality in the Secure Sockets Layer (SSL) operation. Try to understand the functionality of confidentiality and integrity of the SSL operations in the handshake. In this lab, we all analyze the public and private key algorithms with the mode of operations like cipher block chain (CBC), electronic code book (EBC), etc., that helps to understand the attacks chosen cipher attack (CCA)/chosen plaintext attack (CPA).

Observation 2: Message Authentication Code (MAC) and Digital Signatures and certificates and Trust: This gives trust on the SSL and its certificates, how these can be applied and connects two known/unknown users on the in Internet. The primary objective is to understand and design principles of digital certificates and public/private key infrastructures.

Observation 3: To design secure server communications: Main focus of this program is to create certificates. How OpenSSL is used to create certificates? and its requests with the help of symmetric/asymmetric key pairs.

3. Laboratory setup

The computer should install both Linux and Windows. All the computers in the lab need to install latest OpenSSL version.

4. Pre-Preparations to the LAB.

- William Stallings: Cryptography and Network Security.
 - X.509 Certificates
 - Secure Socket Layer and Transport Layer Security
- Additional reading resources
 - Authentication, digital certificates and chains of trust
http://www.mozilla.org/projects/security/pki/psm/help_21/using_certs_help.html
(Using Certificates)
 - Confidentiality and Secure Sockets Layer operation
Man pages for Wireshark
 - Preparing a server for secure communication
<http://www.securityfocus.com/infocus/1818> (Good article about how to set up SSL support for Apache web server, part 1.)
<http://www.securityfocus.com/infocus/1820> (Good article about how to set up SSL

support for Apache web server, part 2.)
<http://www.openssl.org/docs/HOWTO/certificates.txt> (How to create certificates using OpenSSL.)
<http://www.openssl.org/docs/HOWTO/keys.txt> (How to create keys using OpenSSL.)
<http://www.openssl.org/docs/apps/openssl.html> (The OpenSSL application commands manual.)

5 Laboratory assignments

There are three parts of this lab and all parts should be done in the Linux (StuDAT) environment.

Authentication, digital certificates and chains of trust. In the electronic commerce, it is important for clients to be confident that transactions are secure. This means that the client can verify the identity of the e-commerce server and that the transactions between the client and the server are protected from eavesdropping. One way of providing authentication is to use a Public Key Infrastructure (PKI) and to let the e-commerce servers authenticate themselves by providing trusted certificates to the clients. The clients can then benefit from the PKI when verifying the identity of the server. In this assignment we will use a web browser to further investigate the details of certificates and PKI. This assignment requires the use of an ordinary web browser. The instructions are written for the Mozilla Web Browser but any browser of your choice will do as long as it supports certificates.

We will now look into what happens on the user level when a browser connects to a web site providing security. We will focus on events that are visible to the user.

Q1: What are the possible reasons for this security warning?

The security warning holds information about the web server's certificate. Answer the following questions by further investigating the certificate, i.e. click the button "View..." under "Add Exception...", which can be found under "I Understand the Risk".

Definition: OpenSSL is an open-source library containing cryptographic tools.

OpenSSL usage :

Install openssl (or ensure it is installed).

Syntax: *openssl command [options] [arguments]*

In this lab, we will use the following commands:

passwd, enc, genrsa, x509, dgst, req, verify.

Why OpenSSL?

According to the documentation, OpenSSL is a cryptography toolkit implementing the Secure Sockets Layer (SSL) and Transport Layer Security (TLS) network protocols and related cryptography standards required by them.

The openssl program is a command line tool for using the various cryptography functions of OpenSSL's crypto library from the shell. It can be used for

- Creation and management of private keys, public keys and parameters
- Public key cryptographic operations
- Creation of X.509 certificates, CSRs and CRLs
- Calculation of Message Digests
- Encryption and Decryption with Ciphers
- SSL/TLS Client and Server Tests

- Handling of S/MIME signed or encrypted mail
- Time Stamp requests, generation and verification

How to use the command-line tool

The command-line tool openssl provides access to the different tools implemented in the OpenSSL libraries. You can obtain the list of openssl commands with

```
$ openssl help
```

Information about an individual openssl command (e.g. enc) and its arguments can be retrieved with

```
$ openssl enc -help
```

As usual, the detailed information can be found at the corresponding man page

```
$ man openssl
```

The different crypto algorithms implemented in your version of OpenSSL can be listed with

```
$ openssl list-cipher-algorithms  
or with
```

```
$ openssl enc -ciphers
```

Depending on the OpenSSL version.

How to compile your own OpenSSL code

The OpenSSL libraries (libcrypto and libssl) can be easily linked with the command line compiler options *-lcrypto* and *-lssl*.

For instance, you can write your C code and build an application with

```
$ gcc -o myapp myapp.c -lcrypto -lssl
```

1. Playing with stream ciphers in practice

1.1 Using the old RC4 stream cipher

RC4 is still implemented in OpenSSL 1.1.1, although its use is not recommended, but it has been deprecated in version 3.0. Thus, if you have OpenSSL 3.0 installed in your system, you cannot recreate the sample code given in this section. However, it is still worth reading because of the explanations about salted key generation procedures it contains.

Example of openssl call:

```
$ echo -n 'Hello World!' | openssl enc -K '000102030405' -rc4 | xxd
```

where the 12-bytes long plaintext “Hello World!” is encrypted with RC4 (for a default key length of 128 bits) under the secret key ‘000102030405’, represented here in hexadecimal notation and padded with all-zero bytes to complete the required 128 bits. The result is a binary stream that is printed with xxd, producing the output

```
00000000:      a229 a949 2bf1 b974 0a37 dbbf .).I+...t.7..
```

You can decrypt in a similar way with the option -d. In the previous example, you can pipe the output to a new openssl call to recover the original message:

```
$ echo -n 'Hello World!'      | openssl enc -K '000102030405' -rc4 | openssl enc -d -K '000102030405'
-rc4 | xxd
producing the output
```

```
00000000:      4865 6c6c 6f20 576f 726c 6421 Hello World!
```

Using the same key to encrypt several messages is completely insecure, but there is no standard way to introduce some randomness or variability to produce different keystreams under the same secret key. In some old systems a initialization value IV was prepended to the secret key to form the actual RC4 key. For instance a 3-byte IV 01FF03 can be manually prepended to the previous key:

```
$ echo -n 'Hello World!'      | openssl enc -K '01FF03000102030405' -rc4 | xxd
and every new plaintext is encrypted with a different IV. Now, you need the IV value to be able to decrypt
```

the ciphertext. Encrypting instead the all-zero message directly gives you the keystream.

Prepending or appending the IV to the key is insecure in RC4, but there are other options to derive a different RC4 key in every encryption session. For instance, you can use a more involved Key-Derivation Function to transform a secret key (called 'passphrase') and a varying public value (called 'salt') into a (probable) unique key. In OpenSSL you can use this mechanism to encrypt a message. For instance with the passphrase "Not very secure" and the salt 00010203 (in hexadecimal) you can generate an RC4 key and encrypt the same plaintext used before:

```
$ echo -n 'Hello World!'      | openssl enc -k 'Not very secure' -S '00010203' -p -rc4 | xxd
```

This line produces something like

```
00000000:      5361 6c74 6564 5f5f 0001 0203 0000 0000      Salted .....
00000010:      7361 6c74 3d30 3030 3130 3230 3330 3030      salt=00010203000
00000020:      3030 3030 300a 6b65 793d 4545 4633 4642      00000.key=EEF3FB
00000030:      3632 3235 3141 3238 4438 4633 3342 3334      62251A28D8F33B34
00000040:      3337 3445 3741 3636 4431 0a5f 6627 1ad0      374E7A66D1._f'..
00000050:      e74b 3f8c 2bc2 78      .K?+.x
```

containing information about the salt used and the RC4 key computed from the passphrase and the salt, as well as giving the resulting ciphertext. The -p option produces the salt and key information (the salt is padded with zeroes up to the necessary length according to the Key Derivation Function used by openssl). The above output corresponds to openssl version 1.0.2g, while in version 1.1.1d the output is

```
00000000:      5361 6c74 6564 5f5f 0001 0203 0000 0000      Salted .....
00000010:      7361 6c74 3d30 3030 3130 3230 3330 3030      salt=00010203000
00000020:      3030 3030 300a 6b65 793d 4136 4339 4444      00000.key=A6C9DD
00000030:      3545 4336 3331 3833 4632 3946 3234 3032      5EC63183F29F2402
00000040:      4145 4437 3241 3738 4543 0a7c 3110 7543      AED72A78EC.|l.uC
00000050:      4611 c53c 40a7 4e      F..<@.N
```

Without the -p option, the output is only the ciphertext, which includes the 16-bytes long header describing the salt used to derive the key. Using version 1.0.2g, the ciphertext is

```
00000000:    5361 6c74 6564 5f5f 0001 0203 0000 0000    Salted .....
00000010:    5f66 271a d0e7 4b3f 8c2b c278 _f'...K?.+.x
```

Observe that the above outputs generated by different OpenSSL versions reflect different symmetric keys for the same passphrase. This is due to the different Key Derivation Functions (KDF) between OpenSSL versions: KDF used in old versions like 1.0.2g is based on the deprecated MD5 hash function, while it have been updated to the more secure SHA256 hash function in version 1.1.1. Namely, the derived symmetric key in older versions of OpenSSL is just computed as the (truncated) MD5 output of the concatenation of the passphrase and the salt value. Indeed, the output of

```
md5sum <(echo -n 'Not very secure' ; echo -n '0001020300000000' | xxd -p -r)
```

is exactly the key generated above by OpenSSL version 1.0.2g, while the key generated by version 1.1.1d is obtained by replacing md5sum by sha256sum, and truncating the output.

Nowadays, from 2017 on, a more secure iterative construction of a KDF, named PBKDF2, is recommended in order to protect passphrases from dictionary attacks. From OpenSSL version 1.1.1 on this KDF can be enabled with the option -pbkdf2, and the number of iterations can be set with -iter. A number of iterations above 10000 is currently recommended. In addition, a good choice for the salt is a random 128-bit value.

Using the chacha20 stream cipher

The newer chacha20 is a modern stream cipher with a reasonable keysize (256 bits) and a precise way to use an initialization value (64 bits). An iterative mixing algorithm transforms a key, an initialization value and a counter (64 bits) into a keystream block (512 bits). As usual, the keystream blocks are XORed with the message blocks to produce the cipherstream.

The message, key and cipher streams are not byte-oriented, but they use a block size of 512 bits.

The key stream blocks can be generated independently of the others (random-access). In other stream ciphers (like RC4) the keystream has to be computed sequentially.

Chacha is not implemented in earlier versions of OpenSSL, like 1.0.2g, but it is in version 1.1.1d.

```
$ echo -n 'Hello World!'          | openssl enc -K '00' -iv '00' -p -chacha20 -nosalt | xxd
produces
```

```
00000000:    6b65 793d 3030 3030 3030 3030 3030 3030    key=0000000000000000
00000010:    3030 3030 3030 3030 3030 3030 3030 3030    00000000000000000000
00000020:    3030 3030 3030 3030 3030 3030 3030 3030    00000000000000000000
00000030:    3030 3030 3030 3030 3030 3030 3030 3030    00000000000000000000
00000040:    3030 3030 0a69 7620 3d30 3030 3030 3030    0000.iv=00000000
00000050:    3030 3030 3030 3030 3030 3030 3030 3030    00000000000000000000
00000060:    3030 3030 3030 3030 300a 3edd 8cc1 cfd1    0000000000.>.....
00000070:    6aff 3231 0ec4 j.21..
```

Where, the key and the iv values where padded with zero-bytes to the appropriate lengths. A (slightly) more realistic call would be

```
$ echo -n 'Hello World!'          | openssl enc -k 'Not very secure' -S '00010203' -chacha20 | xxd
```

Which uses a passphrase and a value for the salt. The output is now

```
00000000:    5361 6c74 6564 5f5f 0001 0203 0000 0000    Salted .....
00000010:    5593 58dc 45b5 0fd5 25ab 1e57 U.X.E...%..W
```

A (somewhat) practical attack against RC4

The attack scenario

In this section we propose a easy-to-implement key-recovery attack against a particular mode of operation of RC4. We need several assumptions to launch a successful attack:

- The initialization value *iv* is a 3-byte counter prepended to a 13-byte long-term key, making a 16-byte string that is used as the actual RC4 key.
- The *iv* is incremented in every new encryption.
- RC4 is used in a communication system to send some well-structured packets with a constant header of at least one byte *m*[0].
- The attacker has access to many encryptions, so that he can wait for the use of some specific *iv* values.

The previous assumptions can be fulfilled in some systems implementing handshake packets encrypted with RC4.

The attack is based on the following facts:

1. For any byte *x*, using *iv*=01 FF *x* results in a keystream such that its first byte equals *x*+2 with high probability.
2. The first keystream byte produced with *iv*=03 FF *x* is, with a noticeable probability, *x*+6+k[0], where *k*[0] denotes the first byte of the long-term key. Similarly, *iv*=04 FF *x* produces the first keystream byte equal to *x*+10+k[0]+k[1] with a noticeable probability.
3. In general, for *i* ranging from 0 to 12, using *iv*=z FF *x* where *z* is the hexadecimal representation of *i* + 3 often produces the first keystream byte equal to *x*+d[i]+k[0]+k[1]+...+k[i], where *d*[i] is the constant 1 + 2 + . . . + (*i* + 3).

Simulating the attack

In order to simulate the attack, you can first simulate the channel eavesdropping. To do that, start generating a 13-byte random key and a one-byte random message *m*[0] (the first byte of the message suffices to perform the attack, assuming that it is a value fixed by the communication protocol, like a packet length, version number, etc.). Then, for each of the values of *iv* mentioned before, concatenate it with the key and encrypt *m*[0] with the RC4 implementation in OpenSSL. If you have OpenSSL version 3.0, you will need a custom implementation of RC4, like the provided *rc4enc* tool.

Then collect all one-byte ciphertexts *c*[0] and store them along with the corresponding values of the *iv* into a table or file, following the structure

```
0X01FF00 0XDB
0X01FF01 0X60
0X01FF02 0XE8
0X01FF03 0X74
...
```

You can use separate files for every collection of values (one file for the 256 values *iv*=01FF*x*, another file for *iv*=03FF*x*, etc.).

Now you proceed with the actual attack (you now pretend that you do not know the key or the message and you try to learn both from the gathered data).

Using fact 1 described above, compute the 256 values of *c*[0] XOR (*x*+2) and choose the most frequent one as your guess for *m*[0]. Notice that all operations are performed with unsigned bytes, so you must use 8-bit arithmetic (or just take modulo 256 to the result of every computation, so that, for instance, 0XFF+2=0X01).

Tools like sort or unique can be useful to detect the most repeated value. You can also use standard tools (spreadsheets) or a script in your preferred programming language (perl, python, C) to process the collected ciphertext bytes to find the most repeated values and end the simulated attack.

In a similar way, use fact 2 to recover $k[0]$ as the most frequent value of $(c[0] \text{ XOR } m[0]) - x - 6$, where $m[0]$ is the guessed value obtained before.

Compute also the most frequent value of $(c[0] \text{ XOR } m[0]) - x - 10 - k[0]$, where again $m[0]$ and $k[0]$ are the previously guessed values. Remember that each step uses a different collection of values of iv.

You can try to use fact 3 to guess all the remaining bytes of the key.

Lab Practical Work

- (Paper and pencil) From the description of the key scheduling phase of RC4, show facts 1 and 2 described above.
- Implement the attack simulation described above, first choosing a random key and one-byte message, then generating the files containing all the gathered ciphertexts, and finally guessing the secret values from this data.
- Observe that sometimes the guesses are wrong. Compare the guessed values to the original ones.

An example of the trace of a possible attack is the following:

```
key is 7e1a0bbc8c770667be44dce10c and message is 90 Gathering keystream first bytes for IV=01FFxx ...
done Guessing m[0] ... done
Guessed m[0]=90 (with freq. 31) *** OK *** Gathering keystream first bytes for IV=03FFxx ...
done Guessing k[0] ... done
Guessed k[0]=7e (with freq. 16) *** OK *** Gathering keystream first bytes for IV=04FFxx ...
done Guessing k[1] ... done
Guessed k[1]=1a (with freq. 20) *** OK *** Gathering keystream first bytes for IV=05FFxx ...
done Guessing k[2] ... done
Guessed k[2]=0b (with freq. 17) *** OK *** Gathering keystream first bytes for IV=06FFxx ...
done Guessing k[3] ... done
Guessed k[3]=bc (with freq. 13) *** OK *** Gathering keystream first bytes for IV=07FFxx ...
done Guessing k[4] ... done
Guessed k[4]=8c (with freq. 19) *** OK *** Gathering keystream first bytes for IV=08FFxx ...
done Guessing k[5] ... done
Guessed k[5]=77 (with freq. 10) *** OK *** Gathering keystream first bytes for IV=09FFxx ...
done Guessing k[6] ... done
Guessed k[6]=06 (with freq. 12) *** OK *** Gathering keystream first bytes for IV=0AFFxx ...
done Guessing k[7] ... done
Guessed k[7]=67 (with freq. 7) *** OK *** Gathering keystream first bytes for IV=0BFFxx ... done
Guessing k[8] ... done
Guessed k[8]=be (with freq. 12) *** OK *** Gathering keystream first bytes for IV=0CFFxx ...
done Guessing k[9] ... done
Guessed k[9]=44 (with freq. 13) *** OK *** Gathering keystream first bytes for IV=0DFFxx ...
done Guessing k[10] ... done
Guessed k[10]=dc (with freq. 7) *** OK *** Gathering keystream first bytes for IV=0EFFxx ...
done Guessing k[11] ... done
Guessed k[11]=e1 (with freq. 13) *** OK ***

Gathering keystream first bytes for IV=0FFFxx ... done Guessing k[12] ... done
Guessed k[12]=0c (with freq. 10) *** OK ***
```

You have some text files available, files `bytes_01FFxx.dat`, ..., `bytes_0FFFxx.dat`, generated with an unknown random key and plaintext byte. Try to apply the attack to those files and retrieve the first bytes of the key and the ciphertext byte.

You are expected to hand in a short report containing:

- The solutions to the paper and pencil parts.
- A short explanation of how your scripts implementing the attack are organized.
- Some execution traces.
- Your guess of the message and key used to generate the provided text files mentioned above. You must also hand in the scripts implementing the attack.

Integrity based OpenSSL Experiments

Using hash functions in practice

The most commonly used hash functions are probably implemented in any personal computer. In a typical Linux installation, the following command line tools compute the hash digest of a file:

```
$ md5sum foo.dat
```

It uses the hash function MD5 (nowadays considered insecure) to compute the digest of the file `foo.dat` and output the result in a printable (text) format. Similar commands exist for the other common hash functions, like `shasum` or `shasum`, `sha256sum` and `sha3sum`. Probably the last one needs the installation of extra packages.

Another example of use of the previous command is

```
$ echo -n | md5sum
```

that produces the output

```
d41d8cd98f00b204e9800998ecf8427e -
```

Corresponding to the MD5 value of the empty file.

You can also compute the different hash values for a collection of files. For instance,

```
$ md5sum *
```

Will produce a list of all the files in the working directory along with their corresponding hashes. For more details, refer to the man pages of the commands.

You can also use the command line tool `openssl` to compute digests of files. For instance,

```
$ echo -n 'Hello world!' | openssl dgst -md5 -binary | xxd
```

Computes the MD5 value corresponding to the ASCII string “Hello world!”. The result produced by the call to `openssl` is in binary format, and in the example it is processed by `xxd` to produce the following printable output:

```
00000000: 86fb 269d 190d 2c85 f6e0 468c eca4 2a20 ..&.....F...*
```

There are other options of openssl dgst (used instead of -binary) that produces different output formats (e.g., -hex produces a printable hexadecimal output).

You can also select other (more secure) hash functions by replacing -md5 with -sha1, -sha256, etc.

Another way to use hash functions with OpenSSL is directly accessing their implementation in the OpenSSL libraries from your C or C++ programs. In that case, you will need the corresponding C header files, that can be installed in your computer from the OpenSSL development package, with

```
$ sudo apt-get install libssl-dev
```

For instance, the header file for the MD5 implementation can be accessed in your C program with

```
#include <openssl/md5.h>
```

Using Message Authentication Codes in OpenSSL

Some well-known message authentication codes, like CMAC or HMAC, are implemented in OpenSSL. For instance, you can compute the CMAC of a message with the following command:

```
$ mykey=00112233445566778899aabbccddeeff
```

```
$ echo -n 'Hello world!' | openssl dgst -mac cmac -macopt cipher:aes-128-cbc -macopt hexkey:$mykey
```

which produces the output

```
(stdin)= fcae3b5af732a72611ce06b9f680db72
```

The option *-macopt* is used to specify the parameters of CMAC, like the block cipher and the key, with the syntax *-macopt name:value*. The length of the key must match the one required by the specified block cipher.

If you want to compute the CMAC of a file (not the standard input), you only need to provide the filename:

```
$ openssl dgst -mac cmac -macopt cipher:aes-128-cbc -macopt hexkey:$mykey -binary foo.dat | xxd
```

In this example, the call to openssl produces binary output, and it is converted to a printable representation with xxd. The use of HMAC is simpler, because it is the default choice for a MAC algorithm in OpenSSL.

```
$ echo -n 'Hello world!' | openssl dgst -hmac "My secret key"
```

Here, the key is specified, as an ASCII string, in the -hmac option, and there is no restriction on its size (because of the flexible specification of the algorithm HMAC).

The key can be also specified in hexadecimal representation with a syntax similar to the CMAC example:

```
$ echo -n 'Hello world!' | openssl dgst -mac hmac -md5 -macopt hexkey:$mykey
```

Implementing some insecure MACs

Insecure MAC designs like CBC-MAC for arbitrarily long messages or the naïve “one-pass” HMAC (a simplification of the actual HMAC) are not directly implemented in OpenSSL, but they can be built from the implemented hashes and ciphers. The simplified version of HMAC, that computes the tag of a message *m* as only the hash of the concatenation of the key and the message itself, can be implemented with a single call to the command line tool:

```
$ cat key.dat message.dat | openssl -dgst -md5 -binary > tag.dat
```

Here, from the binary files containing the key and the message, the computed tag is saved in the file tag.dat.

Similarly, you can just use encryption in CBC mode to obtain a CBC-MAC implementation, by just selecting the last ciphertext block as the tag. For instance, the AES-128-CBC-MAC tag of the file message.dat with the AES-128 key contained in key.dat can be computed with

```
$ openssl enc -aes-128-cbc -K 'cat hexkey.dat' -iv 0 -in message.dat | tail -c 16 > tag.dat
```

Where a zero initialization value is used, as specified in the definition of CBC-MAC, and the file hexkey.dat contains the hexadecimal representation of the binary file key.dat. You can build this file with

```
$ xxd -p key.dat hexkey.dat
```

The last part of the command line, tail -c 16, takes only the last 16 bytes of the resulting CBC encryption, that corresponds to the last 128-bit block of it.

With these two custom implementation of the one-pass HMAC and CBC-MAC, you can recreate the two well-known length extension forgery attacks.

Lab Practical Work: recreation of known MAC forgery attacks

CBC-MAC concatenation attack

CBC-MAC is insecure if used with variable length messages. Recreate the attack, with the following steps:

- Create a random AES-128 key (which is just a random string of 16 bytes), and choose two arbitrary messages. For instance, message1 is:
“What about joining me tomorrow for dinner?”, and message2 is:
“Oops, Sorry, I just remember that I have a meeting very soon in the morning.”.
Probably, the two messages came from completely different contexts (perhaps with different recipients).
- For simplicity, assume that the system adds a header to the messages consisting of 16 zero bytes. Then, create two files mess1.dat and mess2.dat with the previous text messages, and a file head.dat with 16 zero bytes.
- Generate the corresponding AES-128-CBC-MACs for the two messages with headers and store them in the files tag1.dat and tag2.dat.
- Investigate the padding that AES-128-CBC introduces in the last incomplete block. To do that, you can encrypt a message with AES-128-CBC, and then decrypt the result with the option -nopad. You will recover the padded version of the message (do it with the first message).
The encryption/decryption lines to investigate the padding would be something like

```
$ openssl enc -aes-128-cbc -K $key -iv 0 -in message.dat -out cipher.dat
```

```
$ openssl enc -d -aes-128-cbc -K $key -iv 0 -nopad -in cipher.dat -out padded.dat
```

```
$ xxd padded.dat
```

In the second line, the -d switch means decryption, and -nopad assumes no padding was used and does not discard any byte of the resulting plaintext.

The padding scheme used here is not the one I’ve explained in class. Namely, if there are n missing bytes in the block, then the padding is n bytes all with the value n. If the last block is complete, then add an entire block with all its bytes equal to the block length in bytes (16 or 0x10 in the case of AES-128).

- Create the forgery by appending the files head.dat, mess1.dat, the necessary padding for the first message, the tag tag1.dat and the second message mess2.dat. Store it into forgery.dat. Compute the CBC-MAC of the resulting file and check whether it is exactly the same as tag2.dat.
- You have got a forged message/tag pair.
The file forgery.dat contains some non-printable characters, but if you print it, it probably gives the impression that the forged message is the concatenation of the two original messages, which substantially changes the content of the first one.

In a sort of practical attack you, as the adversary, will be given the plaintexts (including the header) and the two tags, but not the key. Then, the padding for the first message can be easily inferred from the length of the missing part in the last incomplete message block, and the first 16 bytes of the second message must be XORed with tag1. To avoid computing this XOR operation, in this recreation the all-zero header is prepended to the messages (and therefore, XORing is just replacing the header by the tag itself).

One-pass HMAC length extension attack

In a quite similar way, the one-pass HMAC can be easily forged without knowing the key. But the

forgery is even more general than in CBC-MAC. Indeed, the attacker knows only a message `mess1.dat` (no header is necessary here) and the corresponding tag `tag1.dat`. Any string can be appended to `mess1.dat` in a way that the adversary can compute the new valid tag for the concatenation of messages. The only limitation is that the concatenated string must start with the padding that the hash function adds to `mess1.dat`.

The main problem here is the difficulty to continue the Merkle-Damgård chain from scratch. There are two ways to do that:

1. Implement the compression function and the Merkle-Damgård chain (that means implementing the whole hash function).
2. Tweak the code in the OpenSSL library (or use low-level access to it)

The second approach is simpler if you have the development version of the libraries installed (to have access to the C header files), which would require the installation of the package `libssl-dev`.

The computation of a hash function in OpenSSL consists of three steps: initialize, update, finalize. All the internal state of the computation is placed into a context object, that you should not access directly (but you can!). The initialize step sets up the initial values of the internal state of the hash function (like the iv specified in the Merkle-Damgård construction). The update step is called every time new sections of data are added to the hash computation. This updates the internal state. The finalize step is called after the last message block is processed by the update step. Then, the necessary padding is added and the hash value is extracted from the internal state.

For the MD5 hash function, you can access and manually modify the internal state of the hash function. In particular, you can set this internal state as if the MD5 function were in the intermediate state corresponding to any desired point in the computation of the hash of a long message. Moreover, given the MD5 hash value of `mess1.dat` and the total length of the key (the HMAC key) and the message, it is straightforward to reconstruct the last internal state of the computation.

From this point, you can continue performing the update step as many times as you want with the new appended data, and then the finalize step will give you the final hash value. This value is the one-pass HMAC of the concatenation of messages, computed without knowing the key!

In MD5, the padding string takes the form `0x80 0x00 ... 0x00 n0 n1 ... n7`, where the last 8 bytes encode the length in bits of the unpadded message.

The forged message is the concatenation of `mess1.dat`, the padding string and the new appended data. Again, the padding string could add strange nonprintable characters that, in practice, make the forgery look suspicious.

Implementing this attack is challenging because you have to deal with the low-level implementation of the hash function, and also there could be some issues about the byte-order of the integer representation (for the length encoding), which depends on the particular architecture of your computer (integers can be represented in either big-endian or little-endian byte order!).

As an example of how the MD5 initialization tweaking code looks like, the following function is intended to be called just after `MD5_Init(MD5_CTX *)`:

```
void set_ctx(MD5_CTX *pctx, const char *digest, unsigned long nblocks) {
    pctx->A =
        gethexword32(digest);
    pctx->B =
        gethexword32(digest+8);
    pctx->C =
        gethexword32(digest+16);
    pctx->D =
        gethexword32(digest+24);
    nblocks <=& 9; // converting into
    bits pctx->Nh = nblocks>>32;
    pctx->Nl = nblocks&0xFFFFFFFFul;
}
```

where `digest` is a string containing the hexadecimal representation of the MD5 hash value of

mess1.dat, and nblocks is the total number of data blocks (512 bits each) of the concatenation of the key and mess1.dat and the corresponding padding (that in some cases can add an extra block).

The function MD5_LONG gethexword32(const char *) (that also needs to be implemented) reads a 32-bit integer value from its hexadecimal representation. It could be replaced by strtod() but the difference is that gethexword32() reads exactly 8 hexadecimal digits and stops, and the bytes must be interpreted in little-endian order (not in network order: the first byte is the least significant one). For instance, the string "12efc5ca" is interpreted as the number 0xcac5ef12.

Practical work: Building Merkle hash trees

It is quite easy to implement the Merkle hash tree functionality from the previously described tools. All you need is to organize the different calls to the corresponding hash function. Indeed, you only need two types of hash computations:

- Compute the hash of a document doc3.dat with something like

```
$ openssl dgst -sha1 -in doc3.dat -binary > node0.3
```

- Aggregate two existing hashes node2.4, node2.5 into one single value node3.2 with

```
$ cat node2.4 node2.5 | openssl dgst -sha1 -binary > node3.2
```

To prevent some possible attacks that mixes hashes of nodes and documents, you can choose two different prefixes (e.g., 0x35 for documents and 0xE8 for nodes) and prepend the corresponding one to the data to be hashed.

```
$ cat doc.pre doc3.dat | openssl dgst -sha1 -binary > node0.3
```

```
$ cat node.pre node2.4 node2.5 | openssl dgst -sha1 -binary > node3.2
```

Where doc.pre and node.pre contain the selected prefixes.

In the previous examples the following labelling convention is taken (parenthesized expressions like $(i - 1)$ are replaced by their corresponding values):

- Documents to be hashed are named doc0.dat, doc1.dat, ..., doc($n - 1$).dat, assuming there are n documents in total.
- Tree nodes at level i (level 0 corresponds to the leaves) are named node $i.j$, where the hash contained in node $i.j$ is computed from the nodes node $(i - 1).(2j)$ and node $(i - 1).(2j + 1)$.
- Node node $i.j$ is computed only whenever node $(i - 1).(2j)$ exists.
- If node $(i - 1).(2j + 1)$ does not exist, then it is replaced by the empty file in the computation of node $i.j$. Or simply, the missing file is removed from the hash computation, as for example,

```
$ cat node.pre node2.4 | openssl dgst -sha1 -binary > node3.2
```

Each tree layer has about half of the nodes than the previous layer. The root node is obtained when a layer has only one node, and this is the hash value of the collection of documents.

You can add a new node to the tree by just naming it as docn.dat and updating the corresponding nodes (including the root). When n is an exact power of 2, then you will need to add a new layer into the tree with a new root, along with some new nodes from it to the new leave. For instance, if a fifth document doc4.dat is added to the Merkle tree of four documents, then the old root is node2.0 and the root of the new tree is node3.0, and the new nodes node2.1, node1.2 and node0.4 are added to the tree.

On the contrary, removing a document implies recomputing the whole tree from scratch (we will not consider document removal in this implementation).

One interesting feature of Merkle hash trees is that the tree creator stores the whole tree, but she only needs to send the root node to other parties. Then at some point she can convince anyone that a certain document belongs to the collection and it is at a specific position by sending a limited number of node values (the necessary nodes to allow the computation of all the intermediate hashes in the path from the document itself to the root). For instance, in a collection of 4 documents she wants to show that a document test.dat belongs to that collection and it is at the fourth position. The nodes that she must open are node0.2 and node1.0, because then the verifier can compute the values corresponding to node0.3 (the hash of the document, properly prefixed), node1.1 (from node0.2 and node0.3) and node2.0 (from node1.0 and node 1.1), and compare the last with the hash of the collection.

As a practical work, you can try to implement the tree building, the tree updating (only adding a new document) and the generation of the proof of membership and position.

- Start from a list of n arbitrary nodes, and name them doc0.dat, doc1.dat, ...
- Choose two prefixes, one for the computation of the hashes of the documents (leaves of the Merkle tree), and the other for the other tree nodes. You can store the documents and the node files into separate folders (e.g., docs and nodes).
- Write a script or program that on input n it computes all hashes and stores them into the files named node $i.j$ for all suitable values of i and j . Remember that you would need to deal with non-existent nodes (e.g., node $(i - 1).(2j)$ exists but node $(i - 1).(2j + 1)$ does not).
- Create a text file with one line per node, containing in each line the values of i, j and the node hash (i.e., the contents of node $i.j$ in hexadecimal). The file can have a header (e.g., the first line) containing the hash algorithm used (e.g. sha1), the two selected prefixes (in hexadecimal), the number n of documents (decimal number), the depth of the tree (the number of layers including the leaves and the root, as a decimal number) and the root hash (the contents of the tree root in hexadecimal):

```
MerkleTree:sha1:3C3C3C3C:F5F5F5F5:12:5:3f786850e387550fdab836ed7e6dc881de23001
b 0:0:89e6c98d92887913cadf06b2adb97f26cde4849b
0:1:2b66fd261ee5c6cfc8de7fa466bab600bcfe4f69
...
4:0:3f786850e387550fdab836ed7e6dc881de23001b
```

- The first line is the public information of the Merkle tree, and the remaining lines give the private information that allows updating the tree and producing proofs of document membership.
- Write a script or program that inserts a new document in the tree, adding and modifying the necessary nodes and updating the text file describing the tree.
- Write a script or program that produces a proof of membership for a given document doc.dat at position k . The proof consists of a list of nodes that allows the verifier to recompute the hashes in the path from the k -th leave to the root and check that the root value is the same as the given one (the verifier was given in advance the public information of the tree). The proof can be just the corresponding lines of the Merkle tree text file description.
- Write a script or program that verifies the proof given the public information of the Merkle tree (first line of the text file), the document and the proof (text file with the necessary nodes).

Confidentiality Lab Practical Work using OpenSSL

Key Generation with openssl

The openssl tool provides specific commands for key generation of some public key encryption schemes. The command `genpkey` is used to generate a public / secret keypair. The way this command is used depends on the selected public key algorithm.

RSA Keys

```
$ openssl genpkey -algorithm rsa
```

Outputs a text encoding of a public key and a secret key for the RSA encryption scheme, with the default parameters (e.g., the size of the modulus). For example, the previous call results in the output

```
-----BEGIN PRIVATE KEY-----
```

```
MIICdwIBADANBgkqhkiG9w0BAQEFAASCAmEwgGJdAgEAAoGBAKT7B3Vi+hdOPXr5
lljR3Ao5U4JjbdmQ4hhX5hh/uJwEEA7GkKrDTGOqLTcZZgeXH4nrq5SwoG6O4Mi1
hT6s/FSdpRkl6LFNcjpXPT4GtzPycMKhsGu+rnLiJmuXtEMM1Cw3gutYVa63ygJx
f7YDcJxI2FcAd02jsKA11MP6n4CTAgMBAAECgYBfsU8xMli3PeWBN9SEy5zivT+H
6J4lzNinoAxRd3uf2udpepkcwyzCvk19pRi+HFTpza13ED/uAKe3IzqHERVGHFMd
qduffQg5DNDymkqnCcJ5yZLe9qn3tj3/EW3Om3Z2Pvn6R9NLFbN8YnTwjWowFmFe
mR0dYticID6llrv54QJBANqGNGQ2qcDTAeaYeWLY5wnvsm/Apgk3u2qogJB8dp+z
rfey7OAq4B5n+r+0FMZPyqHcs4olUvrCjiEEb/eiL7ECQQDBRh2YpzmW2NBnt+Qy
```

(Generated by lineprocx v2.97)

```
UMrGImRli7GFfp9UKhkRwiOhR1p48mD7yjbSqB7eG0QEVLLN5F02AeALIZDFvTd4
CemDAkEApBc6qDXT6pOITdwY6nztoKx5VSIYhHtxJHo7cEPF385QyDt3XC1V9f8m
b2WOZAvuoPTVbNryIJKPn4NxglYtQQJAHembJQgkmpsdyhl+4OauK3IhNbIkHHfO
WvIU/fGdEBX6A6QHrJCEYABR4RA5O65cKg+A+Z3arhBM4r3BOvvVvwJBAJ2A3l0p
TCNwuGbuUVw8KjJ6zHoLro3pTruhgDZvfjvq+ymFmQ2/o6m0ULiJ2XsUqVnpdzYT
WuOKOkYMxNv3GDo=
```

```
-----END PRIVATE KEY-----
```

The text output is given in PEM (Privacy-Enhanced Mail) format, but you can also produce a human readable text format by adding the option `-text` to the command line:

Private-Key: (1024 bit) modulus:

```
00:a4:fb:07:75:62:fa:17:4e:3d:7a:f9:94:88:d1:
dc:0a:39:53:82:63:6d:d9:90:e2:18:57:e6:18:7f:
b8:9c:04:10:0e:c6:90:aa:c3:4c:63:aa:2d:37:19:
66:07:97:1f:89:eb:ab:94:b0:a0:6e:8e:e0:c8:b5:
85:3e:ac:fc:54:9d:a5:19:25:e8:b1:4d:72:3a:57:
3d:3e:06:b7:33:f2:70:c2:a1:b0:6b:be:ae:72:e2:
26:6b:97:b4:43:0c:d4:2c:37:82:eb:58:55:ae:b7:
ca:02:71:7f:b6:03:70:9c:48:d8:57:00:77:4d:a3: b0:a0:35:d4:c3:fa:9f:80:93
publicExponent: 65537 (0x10001) privateExponent:
5f:b1:4f:31:32:58:b7:3d:e5:81:37:d4:84:cb:9c:
e2:bd:3f:87:e8:9e:25:cc:d8:a7:a0:0c:51:77:7b:
9f:da:e7:69:7a:99:1c:c3:2c:c2:be:49:7d:a5:18:
```

```

be:1c:54:e9:cd:ad:77:10:3f:ee:00:a7:b7:23:3a:
87:11:15:46:1c:53:1d:a9:db:9f:7d:08:39:0c:d0:
f2:9a:4a:a7:09:c2:79:c9:92:de:f6:a9:f7:b6:3d:
ff:11:6d:ce:9b:76:76:3e:f9:fa:47:d3:4b:15:b3:
7c:62:74:f0:8d:6a:30:16:61:5e:99:1d:1d:62:d8:
9c:20:3e:88:96:bb:f9:e1
prime1:
00:da:86:34:64:36:a9:c0:d3:01:e6:98:79:62:d8:
e7:09:ef:b2:6f:c0:a6:09:37:bb:6a:a8:80:90:7c:
76:9f:b3:ad:f7:b2:ec:e0:2a:e0:1e:67:fa:bf:b4:
14:c6:4f:ca:a1:dc:b3:8a:25:52:fa:c2:8e:21:04:
6f:f7:a2:2f:b1 prime2:
00:c1:46:1d:98:a7:39:96:d8:d0:4d:b7:e4:32:50:
ca:c6:22:64:65:8b:b1:85:96:9f:54:2a:19:11:c2:
23:a1:47:5a:78:f2:60:fb:ca:36:d2:a9:be:de:1b:
44:04:54:b2:cd:e4:5d:36:01:e0:0b:95:90:c5:bd:
37:78:09:e9:83
exponent1:
00:a4:17:3a:a8:35:d3:ea:93:88:4d:dc:18:ea:7c:
ed:a0:ac:79:55:29:58:84:7b:71:24:7a:3b:70:43:
c5:df:ce:50:c8:3b:77:5c:2d:55:f5:ff:26:6f:65:
8e:64:0b:ee:a0:f4:d5:6c:da:f2:20:92:8f:9f:83:
71:80:86:2d:41
exponent2:
1d:e9:9b:25:08:24:9a:9b:1d:ca:19:7e:e0:e6:ae:
2b:72:21:35:b2:24:1c:77:ce:5a:f9:54:fd:f1:9d:
10:15:fa:03:a4:07:ac:90:84:61:a0:51:e1:10:39:
3b:ae:5c:2a:0f:80:f9:9d:da:ae:10:4c:e2:bd:c1:
3a:fb:d5:bf coefficient:
00:9d:80:de:5d:29:4c:23:70:b8:66:ee:51:5c:3c:
2a:32:7a:cc:7a:0b:ae:8d:e9:4e:bb:a1:80:36:6f:
7e:3b:ea:fb:29:85:99:0d:bf:a3:a9:b4:50:b8:89:
d9:7b:14:a9:59:e9:77:36:13:5a:e3:8a:3a:46:0c:
c4:db:f7:18:3a

```

In this version of OpenSSL (1.0.2g), the defaults for RSA are a size of 1024 bits and the public exponent 65537. In newer versions the default RSA key size is 2048 bits. The additional fields named exponent1, exponent2 and coefficient are given for optimization of the decryption operation.

You can override the default parameters by providing the desired values via the option -pkeyopt. For instance,

```
$ openssl genpkey -algorithm rsa -pkeyopt rsa_keygen_bits:512 -pkeyopt rsa_keygen_pubexp:17
```

Will use the (insecure) specified values for the key generation.

Diffie-Hellman Keys

Other public key cryptosystems, like the Diffie-Hellman key exchange, require a previous generation of some public parameters.

The same command `genpkey` can be used for public parameter generation with the option `-genparam`. For instance, (the order of the options is important!)

```
$ openssl genpkey -genparam -algorithm dh -pkeyopt dh_paramgen_prime_len:512 -text
```

Produces the output

```
-----BEGIN DH PARAMETERS-----
MEYCQQCp/FMOYEOyuktyyNZea1SR7WZ8E78+j5W7qu+Kwqx6xU4Du5Bvoo+9lK0+
eku5tkScHH3TIMcyLLiTDqldWAprAgEC
-----END DH PARAMETERS-----
```

DH Parameters: (512 bit) prime:
00:a9:fc:53:0e:60:43:b2:ba:4b:72:c8:d6:5e:6b:
54:91:ed:66:7c:13:bf:3e:8f:95:bb:aa:ef:8a:c2:
ac:7a:c5:4e:03:bb:90:6f:a2:8f:bd:94:ad:3e:7a:
4b:b9:b6:44:9c:1c:7d:d3:20:c7:32:2c:b8:93:0e:
a9:5d:58:0a:6b
generator: 2 (0x2)

Corresponding to the PEM and the plain text output formats. Observe that no public / secret keys are

Generated, but only the description of the Diffie-Hellman group. The labels prime and generator can be different across OpenSSL versions.

You can save the PEM output to a file (e.g., providing the option `-out filename`) and use it for actual key generation. This way, you can reuse the same parameters for several keys (e.g., for the two parties in a Diffie-Hellman key exchange).

Given the DH parameter file `mydhgroup.pem`, you can now generate a keypair with

```
$ openssl genpkey -paramfile mydhgroup.pem -text
```

You don't need to provide the name of the public key algorithm or the size parameters, because they can be inferred from the public parameters file header. The output produced by the above command is:

```
-----BEGIN PRIVATE KEY-----
MIGcAgEAMFMGCSqGSIb3DQEDATBGAKEAqfxTDmBDsrpLcsjWXmtUkelmfBO/Po+V
u6rvisKsesVOA7uQb6KPvZStPnpLubZENBx90yDHMiy4kw6pXVgKawIBAgRCAkBx
l8GJCiMPoHx/lZ5I9mNWGlbsRF7gc85d+DXlqNLBSKcwK6KYreCvFFT6ASZTGEuB
t+ON4ZtsEyEQMhnxbokb
-----END PRIVATE KEY-----
```

DH Private-Key: (512 bit) private-key:
71:97:c1:89:0a:23:0f:a0:7c:7f:95:9e:48:f6:63:
56:1a:56:ec:44:5e:e0:73:ce:5d:f8:35:e5:a8:d2:
c1:48:a7:30:2b:a2:98:ad:e0:af:14:54:fa:01:26:
53:18:4b:81:b7:e3:8d:e1:9b:6c:13:21:10:32:19:
f1:6e:89:1b
public-key:
78:1f:c1:e3:e2:7d:22:8d:ad:0a:e2:dd:91:cd:49:
e0:94:19:b0:20:4c:cd:9d:be:68:98:84:a5:9b:96:
5e:c1:46:98:12:c8:e9:ee:f9:d1:48:66:7a:b4:da:

```
fa:46:12:f3:a4:ea:1f:a0:a6:a1:34:54:97:33:d5:
d5:92:2a:21
prime:
00:a9:fc:53:0e:60:43:b2:ba:4b:72:c8:d6:5e:6b:
54:91:ed:66:7c:13:bf:3e:8f:95:bb:aa:ef:8a:c2:
ac:7a:c5:4e:03:bb:90:6f:a2:8f:bd:94:ad:3e:7a:
4b:b9:b6:44:9c:1c:7d:d3:20:c7:32:2c:b8:93:0e:
a9:5d:58:0a:6b
generator: 2 (0x2)
```

Observe that both the prime and the generator are the same as in the given parameters file.

The prime generated with the previous commands is a safe prime (i.e., it has the form $p = 2q + 1$, where q is also prime). This option makes the parameter generation slow and also impacts the performance of the public key operations for these parameters, without a real security improvement.

You can use a faster generation following the X9.42 specification (see RFC2631 for more details), in which $p = kq + 1$ and the prime q can be smaller (e.g., 224 bits long). For that, you need to add the extra option `-pkeyopt dh_paramgen_type:1`, and you can also choose the size q of the DH group with `-pkeyopt dh_paramgen_subprime_len:value`, where `value` must be one of 160, 224 or 256.

For instance, the parameters generated with

```
$ openssl genpkey -genparam -algorithm dh -pkeyopt dh_paramgen_type:1 -pkeyopt \
$ dh_paramgen_prime_len:512 -pkeyopt dh_paramgen_subprime_len:160 -text
```

are

```
-----BEGIN X9.42 DH PARAMETERS-----
MIGdAkEA6Bq3mgJlZJR7ugkcEice6okyZHj4HHiOlsPGn0EFQWWu4wXqZiH3OLcr
MkBaFIZRILHPAeMnKPZ1oxW7Ek2Z5wJBAJHBQ20NsKTeQbeTrVGUG0PYQvFeRoNd
8dHwQRDRHF6tm2ixb/WOqm1xiDffBfduerQlEGx/OLTI/MwMagIIYfYCFQDd9jWt
+nDH56jw49p5ND9bz3OCkQ==
-----END X9.42 DH PARAMETERS-----
```

DH Parameters: (512 bit) prime:

```
00:e8:1a:b7:9a:02:65:64:94:7b:ba:09:1c:12:27:
1e:ea:89:32:64:78:f8:1c:78:8e:96:c3:c6:9f:41:
05:41:65:ae:e3:05:ea:66:21:f7:38:b7:2b:32:40:
5a:14:86:51:94:b1:cf:01:e3:27:28:f6:75:a3:15:
bb:12:4d:99:e7
```

generator:

```
00:91:c1:43:6d:0d:b0:a4:de:41:b7:93:ad:51:94:
1b:43:d8:42:f1:5e:46:83:5d:f1:d1:f0:41:10:d1:
1c:5e:ad:9b:68:b1:6f:f5:8e:aa:6d:71:88:37:df:
05:f7:6e:7a:b4:25:10:6c:7f:38:b4:c8:fc:cc:0c:
6a:02:08:61:f6
```

```
subgroup order: 00:dd:f6:35:ad:fa:70:c7:e7:a8:f0:e3:da:79:34:
3f:5b:cf:73:82:91
```

recommended-private-length: 160 bits

In newer OpenSSL versions (from 3.0 on) the command syntax and the field names in the output are slightly different. The name of the algorithm is now dhx instead of dh. Then the corresponding command will be

```
$ openssl genpkey -genparam -algorithm dhx -pkeyopt dh_paramgen_prime_len:512 \  
$ -pkeyopt dh_paramgen_subprime_len:160 -text
```

You can also use any of the three standardized DH groups (see RFC5114, sections 2.1, 2.2, 2.3) with the option `-pkeyopt dh_rfc5114:value`, where value is one of 1, 2 or 3, as in the example

```
$ openssl genpkey -genparam -algorithm dh -pkeyopt dh_rfc5114:3 -text
```

and again, in newer versions of OpenSSL, you must replace `-algorithm dh` by `-algorithm dhx`. The key files generated with the faster DH groups (either using X9.42 or RFC5114) have some additional fields, e.g., specifying the group order `q`. For example, using the first group in RFC5114, we can build the following keyfile:

```
DH Private-Key: (1024 bit) private-key:  
3d:0f:71:1e:47:44:b2:e1:b7:37:50:01:ea:34:c5: a5:93:91:71:7f  
public-key:  
7e:ee:7b:7e:7f:83:c8:5e:68:1c:6b:6e:43:49:11:  
4f:c3:1d:9a:8c:6e:3c:62:68:d6:fd:ff:06:bc:39:  
21:4f:ec:5a:64:a4:ed:16:6e:1f:07:e4:e6:83:31:  
79:e8:07:8b:61:e0:07:ce:33:12:0b:ec:d9:5a:51:  
0e:62:1c:68:c7:f4:01:53:f8:5b:b1:0f:2d:aa:31:  
1e:39:50:85:83:ff:76:b1:93:f7:67:e5:5c:af:36:  
6a:e7:bf:78:b1:6e:1d:d3:dc:95:60:b5:9f:a1:d0:  
14:ea:b7:eb:a6:3c:be:31:2d:90:b6:e6:2a:c8:f9: 08:43:4e:c4:fc:59:c4:1e  
prime:  
00:b1:0b:8f:96:a0:80:e0:1d:de:92:de:5e:ae:5d:  
54:ec:52:c9:9f:bc:fb:06:a3:c6:9a:6a:9d:ca:52:  
d2:3b:61:60:73:e2:86:75:a2:3d:18:98:38:ef:1e:  
2e:e6:52:c0:13:ec:b4:ae:a9:06:11:23:24:97:5c:  
3c:d4:9b:83:bf:ac:cb:dd:7d:90:c4:bd:70:98:48:  
8e:9c:21:9a:73:72:4e:ff:d6:fa:e5:64:47:38:fa:  
a3:1a:4f:f5:5b:cc:c0:a1:51:af:5f:0d:c8:b4:bd:  
45:bf:37:df:36:5c:1a:65:e6:8c:fd:a7:6d:4d:a7: 08:df:1f:b2:bc:2e:4a:43:71  
generator:  
00:a4:d1:cb:d5:c3:fd:34:12:67:65:a4:42:ef:b9:  
99:05:f8:10:4d:d2:58:ac:50:7f:d6:40:6c:ff:14:  
26:6d:31:26:6f:ea:1e:5c:41:56:4b:77:7e:69:0f:  
55:04:f2:13:16:02:17:b4:b0:1b:88:6a:5e:91:54:  
7f:9e:27:49:f4:d7:fb:d7:d3:b9:a9:2e:e1:90:9d:  
0d:22:63:f8:0a:76:a6:a2:4c:08:7a:09:1f:53:1d:  
bf:0a:01:69:b6:a2:8a:d6:62:a4:d1:8e:73:af:a3:  
2d:77:9d:59:18:d0:8b:c8:85:8f:4d:ce:f9:7c:2a:  
24:85:5e:6e:eb:22:b3:b2:e5  
subgroup order: 00:f5:18:aa:87:81:a8:df:27:8a:ba:4e:7d:64:b7: cb:9d:49:46:23:53
```

or from version 3.0 on:

```
DH Private-Key: (1024 bit) private-key:
3d:0f:71:1e:47:44:b2:e1:b7:37:50:01:ea:34:c5: a5:93:91:71:7f
public-key:
7e:ee:7b:7e:7f:83:c8:5e:68:1c:6b:6e:43:49:11:
4f:c3:1d:9a:8c:6e:3c:62:68:d6:fd:ff:06:bc:39:
21:4f:ec:5a:64:a4:ed:16:6e:1f:07:e4:e6:83:31:
79:e8:07:8b:61:e0:07:ce:33:12:0b:ec:d9:5a:51:
0e:62:1c:68:c7:f4:01:53:f8:5b:b1:0f:2d:aa:31:
1e:39:50:85:83:ff:76:b1:93:f7:67:e5:5c:af:36:
6a:e7:bf:78:b1:6e:1d:d3:dc:95:60:b5:9f:a1:d0:

14:ea:b7:eb:a6:3c:be:31:2d:90:b6:e6:2a:c8:f9: 08:43:4e:c4:fc:59:c4:1e
GROUP: dh_1024_160
```

Elliptic Curve Cryptography

The elliptic curve version of Diffie-Hellman key generation is very similar, but it currently supports only known (standarized) elliptic curve groups (that is, you cannot use openssl to generate new random elliptic curves with the required security properties). In order to create the parameters file for one of the preimplemented elliptic curve groups (e.g., P-256, or equivalently, prime256v1), you simply use

```
$ openssl genpkey -genparam -algorithm ec -pkeyopt ec_paramgen_curve:P-256 -pkeyopt \
$ ec_param_enc:named_curve -text
```

Producing the output

```
-----BEGIN EC PARAMETERS-----
BggqhkjOPQMBBw==
-----END EC PARAMETERS-----
```

```
ECDSA-Parameters: (256 bit) ASN1 OID: prime256v1
NIST CURVE: P-256
```

You can also generate the explicit values of the parameters providing the value explicit to the parameter `ec_param_enc`, like in

```
$ openssl genpkey -genparam -algorithm ec -pkeyopt ec_paramgen_curve:P-256 -pkeyopt \
$ ec_param_enc:explicit -text
```

and the output is now

```
-----BEGIN                               EC                               PARAMETERS-----
MIH3AgEBMCwGBYqGSM49AQECIQD////////AAAAAQAAAAAAAAAAAAAAAAAAP////////
////////zBbBCD////////AAAAAQAAAAAAAAAAAAAAAAAAP////////AQgWsY12Ko6
k+ez671VdpiGvGUdBrDMU7D2O848PifSYEsDFQDnTYIhucEk2pmeOETnSa3gZ9+
kARBBGsX0fLhLEJH+Lzm5WOkQPJ3A32BLeszoPShOUXYmMKWT+NC4v4af5uO5+tK
fA+eFivOM1drMV7Oy7ZAaDe/UfUCIQD////////AAAAAP////////vOb6racXnoTz ucrC/GMIUQIBAQ==
-----END EC PARAMETERS-----
```

ECDSA-Parameters: (256 bit) Field Type: prime-field Prime:

00:ff:ff:ff:ff:00:00:00:01:00:00:00:00:00:00:

00:00:00:00:00:00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:

A:

00:ff:ff:ff:ff:00:00:00:01:00:00:00:00:00:00:

00:00:00:00:00:00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:

B:

5a:c6:35:d8:aa:3a:93:e7:b3:eb:bd:55:76:98:86:

bc:65:1d:06:b0:cc:53:b0:f6:3b:ce:3c:3e:27:d2: 60:4b

Generator (uncompressed): 04:6b:17:d1:f2:e1:2c:42:47:f8:bc:e6:e5:63:a4:

40:f2:77:03:7d:81:2d:eb:33:a0:f4:a1:39:45:d8:

98:c2:96:4f:e3:42:e2:fe:1a:7f:9b:8e:e7:eb:4a:

7c:0f:9e:16:2b:ce:33:57:6b:31:5e:ce:cb:b6:40:

68:37:bf:51:f5

Order:

00:ff:ff:ff:ff:00:00:00:00:ff:ff:ff:ff:ff:ff:ff:

ff:ff:bc:e6:fa:ad:a7:17:9e:84:f3:b9:ca:c2:fc:

63:25:51

Cofactor: 1 (0x1) Seed:

c4:9d:36:08:86:e7:04:93:6a:66:78:e1:13:9d:26:

b7:81:9f:7e:90

Where you can see the values of the prime defining the finite field, the coefficients a, b of the curve, the generator (both the x- and y-coordinates of the point are packed into a single string), the order or size q of the generated group, and other parameters (cofactor and seed).

You can obtain a list of implemented elliptic curves with

```
$ openssl ecparam -list_curves
```

Finally, the generation of a keypair from a given parameters file (e.g., P-128_ecgroup.pem) is exactly as above:

```
$ openssl genpkey -paramfile P-128_ecgroup.pem -text
```

Producing the output

-----BEGIN PRIVATE KEY-----

MIGHAgEAMBMGBYqGSM49AgEGCCqGSM49AwEHBG0wawIBAQQgohhdkTqmahz8bwgh
BbDhmVyaTT4K+f4qatP6/oF8pb+hRANCAAQhmFI+84JDC+eHPmsbDI7qjZlAFDHM
9263g2FZjEs/zZ5W8T6ipYhu+krsX+j8/our09tWrV+5V0L1jkQtcPOb

-----END PRIVATE KEY-----

Private-Key: (256 bit) priv:

00:a2:18:5d:91:3a:a6:6a:1c:fc:6f:08:21:05:b0:

e1:99:5c:9a:4d:3e:0a:f9:fe:2a:6a:d3:fa:fe:81:

7c:a5:bf pub:

04:21:98:52:3e:f3:82:43:0b:e7:87:3e:6b:1b:0e:

5e:ea:8d:99:40:14:31:cc:f7:6e:b7:83:61:59:8c:

4b:3f:cd:9e:56:f1:3e:a2:a5:88:6e:fa:4a:ec:5f:

e8:fc:fe:8b:ab:d3:db:56:ad:5f:b9:57:42:f5:8e:
44:2d:70:f3:9b
ASN1 OID: prime256v1 NIST CURVE: P-256

Where the secret key is an integer modulo the size of the group q , and the public key is a point in the elliptic curve (again, the two coordinates of the point are packed into a single string).

File Formats

In the previous examples you can see the need of packing different mathematical objects together in a single file, and this requires the use of some formatting rules. Along with the human readable text output, OpenSSL uses a printable compressed format PEM and a binary format DER to store keys, ciphertext, signatures and certificates, among other binary objects.

The PEM format is a printable version (using the Base64 printable encoding) of the binary DER format, with additional headers and footers, like for instance:

```
-----BEGIN PRIVATE KEY-----  
...  
(base64 encoded binary DER content)  
...  
-----END PRIVATE KEY-----
```

DER format is basically an unambiguous way to pack a sequence of objects, and each individual object has three parts: type specification, length of the data, and the data itself. For instance, an integer is encoded by as a byte sequence starting with the type identifier 0x02 followed by the number of bytes used in the binary representation of the integer, and then the binary representation itself. The predefined types in DER include booleans, integers, bit strings, octet strings among other types, and aggregations of them. You can use the command `asn1parse` of `openssl` to look inside a DER or a PEM file and to retrieve all individual entities stored into them. In some `openssl` commands the user can choose the specific output format (DER or PEM) to store keys, ciphertexts, certificates and other binary objects. For instance, the option `-outform format`, where `format` is one of `der` or `pem`, used in conjunction with `openssl genpkey` selects the format used to store the generated keypair.

Public Key Extraction

The previous commands show how to generate public / secret keypairs and store them into files. However, in the public key cryptography setting, public keys must be transmitted without revealing the corresponding secret keys. In OpenSSL one can extract the public part of the generated keypair and store it into a separate file. The command

```
$ openssl pkey -in private.pem -pubout -out public.pem
```

Reads the keypair from the file `private.pem`, probably generated with `openssl genpkey`, and extracts and

Stores the public key into the file `public.pem`. Default input and output formats are PEM, but you can specify DER format in either input or output files with the options `-inform der` and `-outform der`.

Now, the resulting public key can be used in encryption or in signature verification, while the private key file is required for decryption and for signature generation. Private key files can be generated in a protected way, by encrypting them with a symmetric key encryption scheme in the same call to `openssl genpkey`. See the `openssl` documentation for more details.

Public Key Encryption and Decryption

From the keypairs generated as explained in the previous section one can perform public key encryption and decryption.

For the (plain) RSA public key encryption scheme the use of OpenSSL in the command line is quite easy, with the `openssl pkeyutl` command used in conjunction with the `-encrypt` and `-decrypt` options.

The main problem is that a public key encryption only allows encrypting messages with a very precise format (e.g., in RSA a message must be the binary encoding of an integer between 1 and $n - 1$, where n is the public modulus). This means that, in practice, we would need a padding scheme, and possibly a chaining mode for long messages.

From a practical point of view, public key encryption schemes are only used to encrypt ephemeral keys to be used in symmetric encryption primitives (like encryption or MAC). Thus, in OpenSSL you can use

```
$ openssl pkeyutl -pubin -inkey myrsapubkey.pem -in ephemeralkey.bin -encrypt -pkeyopt \  
$ rsa_padding_mode:oaep -out ciphertext.bin
```

to encrypt the short file `ephemeralkey.bin` with the RSA key contained in `myrsapubkey.pem` using a specific padding scheme like OAEP (see the documentation of OpenSSL for the implemented RSA padding algorithms), and store the encrypted data in `ciphertext.bin`.

There exist a maximum length for the message to be encrypted (the maximum length could be smaller than the length of n , because the padding string takes some minimal space). Normally, short messages can be encrypted successfully, except if you specify `-pkeyopt rsa_padding_mode:none`, in which only messages with exactly the same binary length of n , and not greater than n as an integer, can be encrypted.

Decryption is performed in a similar way (you also have to specify the same padding scheme used in encryption), but now you provide the private key instead of the public one, and use the option `-decrypt` instead of `-encrypt`.

```
$ openssl pkeyutl -inkey myrsaprivkey.pem -in ciphertext.bin -decrypt -pkeyopt \  
$ rsa_padding_mode:oaep -out decrypted.bin
```

Diffie-Hellman Key Agreement

The generation of the common key in the Diffie-Hellman Key Agreement requires one secret key (say, Alice's secret key) and one public key (say, Bob's public key). Given the files (generated as above) `alice_priv.pem` and `bob_publ.pem`, Alice can generate the common secret with

```
$ openssl pkeyutl -inkey alice_priv.pem -peerkey bob_publ.pem -derive -out common.bin
```

Similarly, Bob can execute

```
$ openssl pkeyutl -inkey bob_priv.pem -peerkey alice_publ.pem -derive -out common.bin
```

To obtain the same result.

According to the specification of the Diffie-Hellman Key Agreement protocol, the two keypairs must be produced with the same group (i.e., the same parameters file). If the group corresponds to the normal (i.e., non elliptic curve based) Diffie-Hellman Agreement, then the common secret is the binary representation of $g^{s_A s_B}$, where s_A and s_B are the secret keys of the two parties, and g is the group generator. Otherwise, in the elliptic curve based protocol, the common secret is the binary representation of the x-coordinate of the point $(s_A s_B)G$, where G is the base point (generator) of the subgroup.

The common secret must never be directly used as a symmetric key. Instead, it must be hashed or transformed with a secure key derivation function. For instance, you can pipe the previous generation commands and the hash computation with

```
$ openssl pkeyutl -inkey alice_priv.pem -peerkey bob_publ.pem -derive | openssl dgst \
$ -sha256 -binary > commonkey.bin
```

Which produces a secure shared 256 bit symmetric key, ready to be used with any encryption or MAC operation.

Lab Practical Work: Hashed ElGamal + Hybrid Encryption + HMAC

As a practical exercise you can implement a version of Hashed ElGamal encryption scheme combined with a symmetric encryption scheme and a MAC.

We use a normal DH group, with a size of 256 bits. The common secret resulting from a Diffie-Hellman agreement is 256 bits long. We use SHA256 to obtain a 256 bits symmetric key, that is splitted into two 128 bits keys: one, k1, for AES-128-CBC encryption and the other, k2, for SHA256-HMAC.

Before encrypting a file, the recipient must have been generated a parameter file for the DH group and a long-term Diffie Hellman keypair, and then she must have send us the corresponding public key. Namely, she needs to perform the following steps:

- Generate the DH group, or choose one of the predefined standard groups. For instance, she chooses the third group in RFC5114 and generates the corresponding parameters file with

```
$ openssl genpkey -genparam -algorithm dh -pkeyopt dh_rfc5114:3 -out param.pem
```

(from version 3.0 on, you need to replace -algorithm dh with -algorithm dhx)

- Generate the long-term keypair with

```
$ openssl genpkey -paramfile param.pem -out alice_pkey.pem
```

- Extract the long-term public key with

```
$ openssl pkey -in alice_pkey.pem -pubout -out alice_pubkey.pem
```

- Publish the files param.pem and alice_pubkey.pem. Actually, she would need to obtain a certificate for this public key form a certification authority to prevent impersonation attacks.
- Now, the encryption operation consists of:
- Compute an ephemeral Diffie-Hellman keypair $r, a = gr$ with openssl genpkey using the file param.pem published by the recipient, and store it in the file ephpkey.pem.
- Generate the corresponding ephemeral public key file (in PEM format) with openssl pkey and store it in the file ephpubkey.pem.
- Derive a common secret from the secret ephemeral key r , contained in ephpkey.pem, and the long-term public key of the recipient, contained in alice_pubkey.pem, with openssl pkeyutl with the -derive option.
- Apply SHA256 to the common secret with openssl dgst (see the previous practical work), and split it into two halves to obtain k1 and k2. Starting from the 32 bytes long binary file with the hash value, you can use head -c 16 and tail -c 16 to extract the first and the last 16 bytes.
- Encrypt the desired file with AES-128-CBC using key k1, with openssl enc -aes-128-cbc and store the result in the file ciphertext.bin (see previous practical works). Some useful tools helps converting binary files into the plain hexadecimal representation, like xxd -p. You would need to provide an iv for the encryption operation. You can generate a random one with openssl rand 16 and store it in the file iv.bin.

- Use key k2 to compute the SHA256-HMAC tag of the concatenation of iv.bin and ciphertext.bin with openssl dgst -hmac -sha256 to obtain the binary file tag.bin (again, you can find the details in the previous practical work).
- The final ciphertext consists of the four files ephpubkey.pem, iv.bin , ciphertext.bin and tag.bin.
- Ideally, a single file in PEM or DER format joining the four would be a better option. You can do that by concatenating the printable (base64 encoded) files with adequate headers and footers. For instance, you can start from the file ephpubkey.pem that already is in PEM format, then concatenate it with the base64 encoding of ciphertext.bin , iv.bin and tag.bin with some PEM style headers and footers added:

```
(Put here the file ephpubkey.pem as it is)
(it already contains its own header and footer lines)
-----BEGIN AES-128-CBC IV-----
(put here the 64bit encoding of file iv.bin)
-----END AES-128-CBC IV-----
-----BEGIN AES-128-CBC CIPHERTEXT-----
(put here the 64bit encoding of file ciphertext.bin)
-----END AES-128-CBC CIPHERTEXT-----
-----BEGIN SHA256-HMAC TAG-----
(put here the 64bit encoding of file tag.bin)
-----END SHA256-HMAC TAG-----
```

To convert files between binary and base64 encoding you can use the special OpenSSL pseudocipher in openssl enc with the -a option. In the encryption direction the input is interpreted as a binary file and the output is the corresponding printable base64 encoding. In the decryption direction (with the -d option), the conversion is done in the opposite way.

Decryption can be performed with a similar procedure:

- Parse the ciphertext file and obtain the four components ephpubkey.pem, iv.bin , ciphertext.bin and tag.bin.
- Use files alice_pkey.pem and ephpubkey.pem to recover the common secret with openssl pkeyutl -derive.
- Apply SHA256 to the previous result and split the value into the two keys k1 and k2.
- Recompute the SHA256-HMAC from the concatenation of the files iv.bin and ciphertext.bin using the key k2. If the result is different from the file tag.bin, then abort the decryption operation and report the error.
- Use openssl enc -d -aes-128-cbc with the key k1 and the iv given in iv.bin to decrypt the file ciphertext.bin and recover the plaintext.

In a complete system you would provide three scripts: one for parameter and key generation, another for encryption and a third one for decryption.

Summery and Additional Lab Assignments to Learn OpenSSL

Encoding with base64 Definition:

Base64 is an encoding scheme which uses 65 printable characters (26 lower-case letters, 26 upper-case letters, 10 digits, characters '+' and '/', and special character '='). Base64 allows to exchange data with limited encoding problems.

Syntax: To encode with base64, the following command is used:

openssl enc -base64 -in input-file -out output-file

To decode with base64, the following command is used:

openssl enc -base64 -d -in input-file -out output-file

- Encode a text file containing an arbitrary password with base64, and send it to your lab partner. Your lab partner has to find your password from this file.
- Is base64 a safe way to protect a password?

Password files Definition:

File /etc/passwd contains information on users. By default, the file is readable by every user.

man 5 passwd

Definition: File /etc/shadow contains information on the password of users. By default, this file is readable only by the super-user.

man 5 shadow

Create an user user1 with password pass1 (for instance) using the adduser command.

man 8 adduser

In file /etc/shadow, identify the field containing the password of the user. Note that the password is encrypted.

Definition: The password can be encrypted using DES or MD5. If the field containing the password starts with \$1\$, the password is encrypted using MD5, otherwise it is encrypted with DES. The salt (which is an additionnal randomness) used for the password occurs before the next \$. Finally, the encrypted password follows the last \$.

Syntax: *openssl passwd [options]*

Where the possible options are:

-crypt: to encrypt

-1: to change the standard encryption algorithm from DES to MD5

-salt salt: to add the salt.

- From the (known) password and the salt from /etc/shadow, obtain the field containing the encrypted password using openssl.
- Change the pasword from user user1 and use only four lower-case letters. Implement a program that takes as input the encrypted password and the salt from /etc/shadow, and that finds out (using brute force) the password. How long does it take to crack the password?
- Would it be possible to adapt your program so that the salt is no longer required? How long would it take to run?

Encryption

Syntax: To encrypt, we can use command `enc`. To decrypt, we can use command `enc` with option `-d`. To use DES, we can use option `-des`. To use triple-DES, we can use option `-des3`. We will also use option `-nosalt` in the following.

- Encrypt an arbitrary file and decrypt it using the good password (with DES and no salt).
- Encrypt an arbitrary file and try to decrypt it using a wrong password. Remark: openssl detects that the password was wrong. We will try to see how openssl could detect it.
- Compare the size of a plaintext and the corresponding cipher. Explain the difference.
- Consider two different files `plaintext1` and `plaintext2` (with different sizes). Encrypt these two files with a password. You obtain files `cipher1` and `cipher2`. Decrypt these two files with *option -nopad*. You obtain `plaintext1nopad` and `plaintext2nopad`.
- With an hexadecimal editor (such as tool `xxd`), study the difference between `plaintext1` and `plaintext1nopad`, and between `plaintext2` and `plaintext2nopad`. Explain the differences.
- Consider another file `plaintext3`. Encrypt it, and ask your lab partner to decrypt it using option `-nopad` (either with a good or a wrong password) into a file `password3nopad`. Without comparing `plaintext3` and `plaintext3nopad`, can you tell whether the password was good or not?

Encryption with RSA Key generation Syntax:

- (a) The key generation is performed using:
`openssl genrsa -out key.pem size`
- (b) Create a private RSA key of 50 bits, and save it into file `mySmall.pem`.
- (c) Syntax: To obtain information on a key, we can use:
`openssl rsa -in key.pem -text -noout`
- (d) Display information on the private key you generated.
- (e) Should the private key be stored as plaintext? Is your key safely stored in file `mySmall.pem`? Break the following 50-bits RSA key `otherSmall.pem` by finding the two factors of `n` (called the modulus)
-----BEGIN RSA PRIVATE KEY-----
MDcCAQACBwLXEvQUSR0CAwEAAQIHAf9IXGeOgQIEAcx/UQIEAZQyDQIEAK3/cQIEAKWD/QIDPAMq -----END RSA PRIVATE KEY-----
- Crack your own RSA key by factorizing the modulus. Why could you crack it? Syntax: In order to save the key in a safe way, a symmetric encryption algorithm (such as triple-DES) can be used:
`openssl genrsa -des3 -out private-key size`
- Create a private key `private1.pem` of 1024 bits, without encryption. Create a private key `private2.pem` of 1024 bits, with encryption.
- Retrieve the information from keys `private1.pem` and `private2.pem`.
Syntax: To retrieve the public key associated with a private key, we use:
`openssl rsa -in private-key -pubout -out public-key`
- Create a public key associated to the private keys `private1.pem` and `private2.pem`.
- Should we store the public key in plaintext?
Syntax: To display information on a public key, we have to use option `-pubin` in order to inform openssl that the input file only contains public data. Retrieve the information of the two public keys

Encryption and decryption Syntax:

- a) To encrypt using a public key, we use:
openssl rsautl -encrypt -in plaintext -inkey public-key -pubin -out cipher
 - b) To decrypt, we use option
-decrypt.
 - c) Encrypt and decrypt a small file.
 - d) Encrypt a large file. What happens? Why?
 - e) Ask your lab partner for his/her public key. Encrypt a short message with his/her public key. Give the cipher to him/her, and ask him/her to decrypt it.
Syntax: To encrypt using DES, we use:
openssl enc -des -in plaintext -out cipher
 - f) To decrypt with DES, we use:
openssl enc -des -d -in cipher -out plaintext
- Ask your lab partner for his/her public key. Encrypt a long message with a symmetric encryption algorithm (such as DES), by using an arbitrary password. Encrypt the password with the public key of your lab partner. Send to your lab partner both the encrypted password and the cipher. Ask him/her to decrypt the message.

Signatures

- g) Syntax: To hash a file, we use:
- h) *openssl dgst -md5 -out hash file*
- i) Syntax: To sign a hash, we use:
- j) *openssl rsautl -sign -in hash -inkey key -out signature*
- k) Should we use the public key or the private key for a signature? Syntax: To verify a signature, we use: *openssl rsautl -verify -in signature -pubin -inkey public-key -out hash2* Exercise 5.16: Sign a message.
- l) Verify the signature. You can use the tool *diff*.
- m) Create three messages. Sign all of them. Slightly modify one or two of them, and send them to your lab partner, together with the signatures. Ask him/her to determine which messages were modified.

Certificates

Certificate generation

- a) Syntax: To generate a certificate, a request has to be created first:
openssl req -new -key key -out request
Remark: The key has to be large enough to sign the certificate. Generally, a 1024-bit key should be sufficient.
- b) Create a certificate request called *user-request.pem*, concerning an entity called *user*. This request is waiting to be signed by a certification authority.
Syntax: To visualize a certificate request or a certificate, we use:
openssl x509 -in certificate -text -noout
- c) Visualize the content of the certificate request.
Create a pair of keys for the certification authority. Create a certificate request called *carequest.pem* and concerning the certificate authority (referred to as CA).

Syntax: To autosign a certificate request, we use:
`openssl x509 -req -in request -signkey private-key -out certificate`

d) Autosign the certificate request of CA in a certificate called `ca-certificate.pem`.

e) Visualize the certificate `ca-certificate.pem`.

Syntax: To sign a certificate request, we use:

`openssl x509 -days duration -CAserial serial -CA certificate -CAkey key -in request -req -out certificate`

The file `serial` contains a number stored in hexadecimal (using an even number of hexadecimal digits).

f) Sign the certificate request `user-request.pem` to produce a certificate `user-certificate.pem`.

Syntax: To verify a certificate, use:

`openssl verify -CAfile ca-certificate certificate`

g) Verify the certificate `user-certificate.pem`.

h) Modify the content of the certificate `user-certificate.pem` and try to verify it.

Certification chain

- Create two certification authorities CA1 and CA2. CA1 is the root certification authority, and CA2 is certified by CA1.
- Create a certificate request and sign it by CA2.
- Verify the complete certification chain.

Assignment on OpenSSL (for 32 bit OS) you may change to 64 bit

Generate two files like in the following.

file.txt

```
0000000000000000
1111111111111111
2222222222222222
0000000000000000
1111111111111111
2222222222222222
0000000000000000
1111111111111111
2222222222222222
```

modified_file.txt (only one DIGIT change)

```
0000000000000000
1111111111111111
2222222222222222
0000000100000000
```

```
1111111111111111
2222222222222222
0000000000000000
1111111111111111
2222222222222222
```

Use AES encryption and mode of operation ECB and CBC to analyze the cipher text.

Use syntax like in the following

```
$ openssl aes-256-ecb -e -in file.txt -out cipher_ecb.bin -nosalt
```

After use

```
$xxd cipher_ecb.bin
```

Observe the output you will be able to understand the known plaintext

Same way find another mode of operation CBC.

```
$ openssl aes-256-cbc -e -in file.txt -out cipher_cbc.bin -nosalt
```

After use

```
$xxd cipher_cbc.bin
```

Find the observations